

Aula rápida — Métodos Básicos de Ordenação

Estruturas de Dados e Algoritmos — Python

Duração sugerida: 60–80 minutos **Pré-requisitos:** vetores/listas, for, while, funções e Big-O.

1. Questão inicial — 5 min

```
numeros = [8, 3, 5, 1, 9, 4]
```

Como podemos transformar essa lista em **[1, 3, 4, 5, 8, 9]** sem utilizar **sort()**?

A conexão com a aula de buscas é direta: a busca binária é $O(\log n)$, mas depende de dados ordenados.

2. O que significa ordenar? — 5 min

Ordenar significa reorganizar elementos segundo determinado critério.

Crescente: 1, 3, 5, 7, 9

Decrescente: 9, 7, 5, 3, 1

Alfabética: Ana, Bruno, Carlos, João

Algoritmo	Ideia principal	Pior caso
Bubble Sort	Trocar vizinhos	$O(n^2)$
Selection Sort	Selecionar o menor	$O(n^2)$
Insertion Sort	Inserir na posição correta	$O(n^2)$

Os três algoritmos chegam ao mesmo resultado usando estratégias diferentes.

3. Bubble Sort — 15 min

Ideia: compara elementos vizinhos e troca suas posições quando estão na ordem errada.

```
[5, 3, 8, 2]
```

```
5 e 3 → [3, 5, 8, 2]
```

```
5 e 8 → [3, 5, 8, 2]
```

```
8 e 2 → [3, 5, 2, 8]
```

Ao final da primeira passagem, o maior elemento, 8, chegou ao final. O processo é repetido até a lista ficar ordenada.

```
def bubble_sort(lista):
    n = len(lista)

    for i in range(n):
        for j in range(n - 1 - i):
            if lista[j] > lista[j + 1]:
                lista[j], lista[j + 1] = lista[j + 1], lista[j]

    return lista

numeros = [5, 3, 8, 2]
print(bubble_sort(numeros))
```

Resultado: **[2, 3, 5, 8]**. O trecho fundamental compara vizinhos: `lista[j] > lista[j + 1]`.

4. Selection Sort — 15 min

Em vez de trocar vizinhos repetidamente, o Selection Sort procura o menor elemento da região ainda não ordenada e o coloca na próxima posição definitiva.

```
[5, 3, 8, 2]
↓ menor = 2
```

```
[2, 3, 8, 5]
  ↑ posição definitiva

def selection_sort(lista):
    n = len(lista)

    for i in range(n):
        menor = i

        for j in range(i + 1, n):
            if lista[j] < lista[menor]:
                menor = j

        lista[i], lista[menor] = lista[menor], lista[i]

    return lista
```

A variável **menor** guarda o **índice** do menor valor encontrado, e não o valor em si.

5. Insertion Sort — 15 min

O Insertion Sort pode ser comparado à organização de cartas na mão: mantém uma região já ordenada e insere cada novo elemento na posição apropriada.

```
8
3 8
3 5 8
2 3 5 8

def insertion_sort(lista):
    for i in range(1, len(lista)):
        atual = lista[i]
        j = i - 1

        while j >= 0 and lista[j] > atual:
            lista[j + 1] = lista[j]
            j -= 1

        lista[j + 1] = atual

    return lista

[3, 5, 8 | 2]
----- ↑
ordenado  inserir
```

Os valores maiores são deslocados para a direita até surgir a posição correta para o elemento atual.

6. Comparação dos três — 10 min

```
numeros = [7, 4, 9, 2, 5, 1]
```

Resultado dos três métodos:

```
[1, 2, 4, 5, 7, 9]
```

- **Bubble Sort:** compara vizinhos, faz trocas e conduz os maiores para o final.
- **Selection Sort:** procura o menor, seleciona e coloca na próxima posição.
- **Insertion Sort:** mantém uma região ordenada e insere o próximo elemento no lugar correto.

7. Complexidade

Nos três algoritmos básicos aparece, no pior caso, um comportamento aproximadamente proporcional a $n \times n$, portanto:

$O(n^2)$

n	n	n ²
10	10	100
100	100	10.000

1.000	1.000	1.000.000
10.000	10.000	100.000.000

Se dobramos o tamanho da entrada, o trabalho de um algoritmo quadrático pode crescer aproximadamente quatro vezes. Algoritmos como Merge Sort e Quick Sort podem apresentar comportamento da ordem de $O(n \log n)$.

8. Atividade prática — Comparando os algoritmos

```
numeros = [
    38, 12, 45, 7, 29,
    18, 41, 3, 25, 10
]
```

Execute manualmente **bubble_sort()**, **selection_sort()** e **insertion_sort()**. Todos devem produzir:

```
[3, 7, 10, 12, 18, 25, 29, 38, 41, 45]
```

Depois, modifique cada algoritmo para contar comparações e trocas/movimentações.

```
comparacoes = 0
trocas = 0

# a cada comparação:
comparacoes += 1

# quando houver troca:
trocas += 1
```

O objetivo é perceber que algoritmos pertencentes à mesma classe assintótica podem apresentar comportamentos concretos diferentes.

9. Desafio rápido

```
ordenada = [1, 2, 3, 4, 5, 6, 7, 8]

invertida = [8, 7, 6, 5, 4, 3, 2, 1]

quase_ordenada = [1, 2, 3, 5, 4, 6, 7, 8]
```

Execute os três algoritmos e compare as operações. Questão: **o estado inicial dos dados interfere igualmente nos três algoritmos?**

A discussão introduz melhor caso, caso médio, pior caso, adaptabilidade, estabilidade, custo de comparações e custo de movimentações.

10. Fechamento da aula

```
BUBBLE      → "Compare vizinhos"
SELECTION   → "Encontre o menor"
INSERTION    → "Insira no lugar certo"

Bubble Sort  →  $O(n^2)$ 
Selection Sort →  $O(n^2)$ 
Insertion Sort →  $O(n^2)$ 
```

Pergunta para continuidade: **se todos esses algoritmos ficam caros quando n cresce, como podemos ordenar milhões de elementos eficientemente?**

Progressão natural: divisão e conquista → Merge Sort → Quick Sort → comparação experimental entre $O(n^2)$ e $O(n \log n)$.

Complemento — O método `sort()` do Python

1. O que é `sort()`?

`sort()` é um método das listas do Python que ordena os elementos da própria lista.

```
numeros = [8, 3, 5, 1, 9, 4]

numeros.sort()

print(numeros)
```

Resultado: **[1, 3, 4, 5, 8, 9]**. A característica importante é que **`sort()`** modifica a própria lista.

```
ANTES: [8, 3, 5, 1, 9, 4]
        sort()
        ↓
DEPOIS: [1, 3, 4, 5, 8, 9]
```

2. `sort()` não é um algoritmo

`numeros.sort()` é uma interface fornecida pelo Python para solicitar uma ordenação. Internamente, o Python executa uma implementação de ordenação altamente otimizada.

O Python utiliza uma família de algoritmo chamada **Timsort**, construída a partir de ideias de Merge Sort e Insertion Sort, além de diversas otimizações.

```
bubble_sort(numeros)    # algoritmo implementado por nós
selection_sort(numeros) # algoritmo implementado por nós
insertion_sort(numeros) # algoritmo implementado por nós

numeros.sort()          # implementação otimizada do Python
```

3. Por que não usar simplesmente `sort()`?

Em programas reais, normalmente devemos preferir **`sort()`** em vez de reinventar um algoritmo de ordenação. Em Estruturas de Dados e Algoritmos, entretanto, implementamos os métodos manualmente para compreender:

- como ocorre a ordenação;
- quantas comparações são necessárias;
- quantas movimentações são realizadas;
- como estratégias diferentes resolvem o mesmo problema;
- por que $O(n^2)$ se torna problemático;
- por que algoritmos mais sofisticados são necessários.

É semelhante a aprender multiplicação antes de usar uma calculadora.

4. `sort()` × `sorted()`

`sort()` altera a lista original:

```
numeros = [5, 2, 8, 1]
numeros.sort()
print(numeros)

# [1, 2, 5, 8]
```

`sorted()` cria uma nova lista ordenada, preservando a original:

```
numeros = [5, 2, 8, 1]
ordenados = sorted(numeros)

print(numeros)
print(ordenados)

# [5, 2, 8, 1]
# [1, 2, 5, 8]
```

Recurso	Modifica a lista original?	Retorna lista ordenada?
lista.sort()	Sim	Não
sorted(lista)	Não	Sim

Uma pegadinha importante:

```
numeros = [5, 2, 8, 1]
resultado = numeros.sort()
print(resultado)

# None
```

Isso ocorre porque **sort()** ordena *in-place* e não devolve a lista ordenada como valor de retorno.

5. Ordem decrescente

```
numeros.sort(reverse=True)
```

O parâmetro **reverse=True** solicita ordenação em ordem decrescente.

6. Critério de ordenação com key

O parâmetro **key** permite determinar o valor utilizado como critério de comparação.

```
nomes = ["Ana", "Alexandre", "João", "Lu"]

nomes.sort(key=len)

print(nomes)

# ['Lu', 'Ana', 'João', 'Alexandre']
```

Nesse caso, o Python usa o comprimento de cada string como chave de ordenação.

7. Complexidade do sort()

Método	Complexidade de referência
Bubble Sort	$O(n^2)$
Selection Sort	$O(n^2)$
Insertion Sort	$O(n^2)$ no pior caso
Python sort() / Timsort	$O(n \log n)$ no pior caso

O Timsort também consegue aproveitar trechos que já estão ordenados, podendo aproximar-se de $O(n)$ em entradas muito favoráveis.

8. Síntese

Se Python já possui sort(), por que estudar algoritmos de ordenação?

Porque **sort()** resolve o problema para o programador; estudar os algoritmos explica por que algumas formas de resolvê-lo são muito melhores que outras.